Foundations of R Software

Lecture 36

Strings – Display and Formatting

:::

Print and Format with Concatenate

Shalabh

Department of Mathematics and Statistics

Indian Institute of Technology Kanpur

NPTEL

Formatting and Display of Strings

Suppose we want to print

```
The zero occurs at 2*pi radians.
```

NPTEL

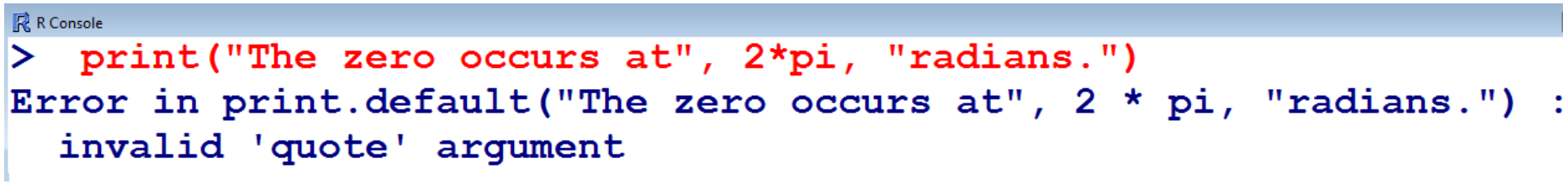
Formatting and Display of Strings

The `print` function has a significant limitation that it prints only one object at a time.

Trying to print multiple items gives error message:

```
>  print("The zero occurs at", 2*pi, "radians.")
Error in print.default("The zero occurs at", 2 *
pi, "radians.") :  invalid 'quote' argument
```

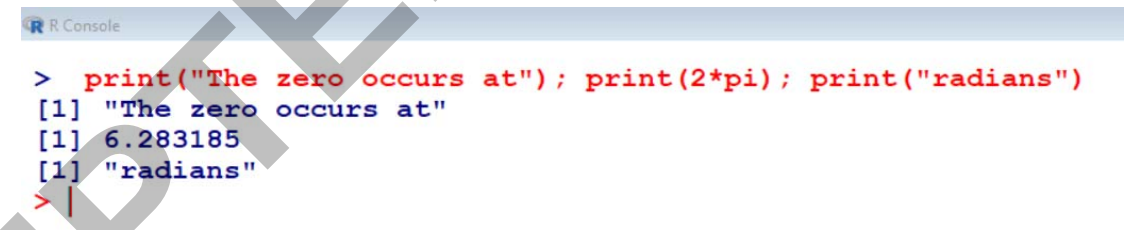
NPTEL



Formatting and Display of Strings

The only way to print multiple items is to print them one at a time

```
> print("The zero occurs at"); print(2*pi);
print("radians")
[1] "The zero occurs at"
[1] 6.283185
[1] "radians"
```



NPTEL

The `cat` function is an alternative to print that lets you combine multiple items into a continuous output.

Formatting and Display of Strings : cat function

- The function `cat()` concatenate (link in the same sequence) and print the arguments in strings, concatenates them and prints the entire string in the command window.

- `cat` is useful for producing output in user-defined functions.

- It converts its arguments to character vectors, link them together in the same sequence to a single character vector, appends the given `sep = string(s)` to each element and then outputs them.

NPTEL

Formatting and Display of Strings : cat function

Usage

```
cat(... )
cat(... , file = "", sep = " ", fill = FALSE,
labels = NULL, append = FALSE)
```

`cat` **puts a space between each item by default.**

One must provide a newline character (`\n`) (newline) to terminate the line.

NPTEL

Formatting and Display of Strings : cat function

`fill` : a logical or (positive) numeric controlling how the output is broken into successive lines. If FALSE (default), only newlines created explicitly by `"\n"` are printed. Otherwise, the output is broken into lines with print width equal to the option width if fill is TRUE, or the value of fill if this is numeric. Non-positive fill values are ignored, with a warning.

`labels` : character vector of labels for the lines printed. Ignored if fill is FALSE.

`append` : logical. Only used if the argument file is the name of file. If TRUE output will be appended to file; otherwise, it will overwrite the contents of file.

NPTEL

Formatting and Display of Strings: print vs cat

The only way to print multiple items is to print them one at a time

```
> print("The zero occurs at"); print(2*pi);
print("radians")
[1] "The zero occurs at"
[1] 6.283185
[1] "radians"
```

The `cat` function is an alternative to print that lets you combine multiple items into a continuous output:

```
> cat("The zero occurs at", 2*pi, "radians.",
"\n")
The zero occurs at 6.283185 radians.
```

NPTEL

Formatting and Display of Strings

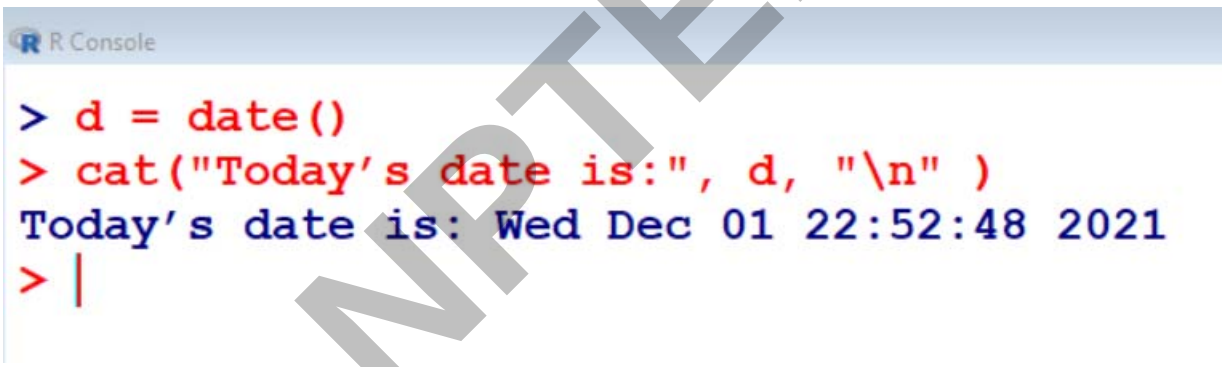
R Console

```
> print("The zero occurs at"); print(2*pi); print("radians")
[1] "The zero occurs at"
[1] 6.283185
[1] "radians"
>
> cat("The zero occurs at", 2*pi, "radians.", "\n")
The zero occurs at 6.283185 radians.
```

NPTEL

Formatting and Display of Strings

```
> d = date()
> cat("Today's date is:", d, "\n" )
Today's date is: Wed Dec 01 22:52:48 2021
```



NPTEL

[Please note this is the time at the time or preparation of slide]

Formatting and Display of Strings

```
> x = 1:10
> x
 [1]  1  2  3  4  5  6  7  8  9 10

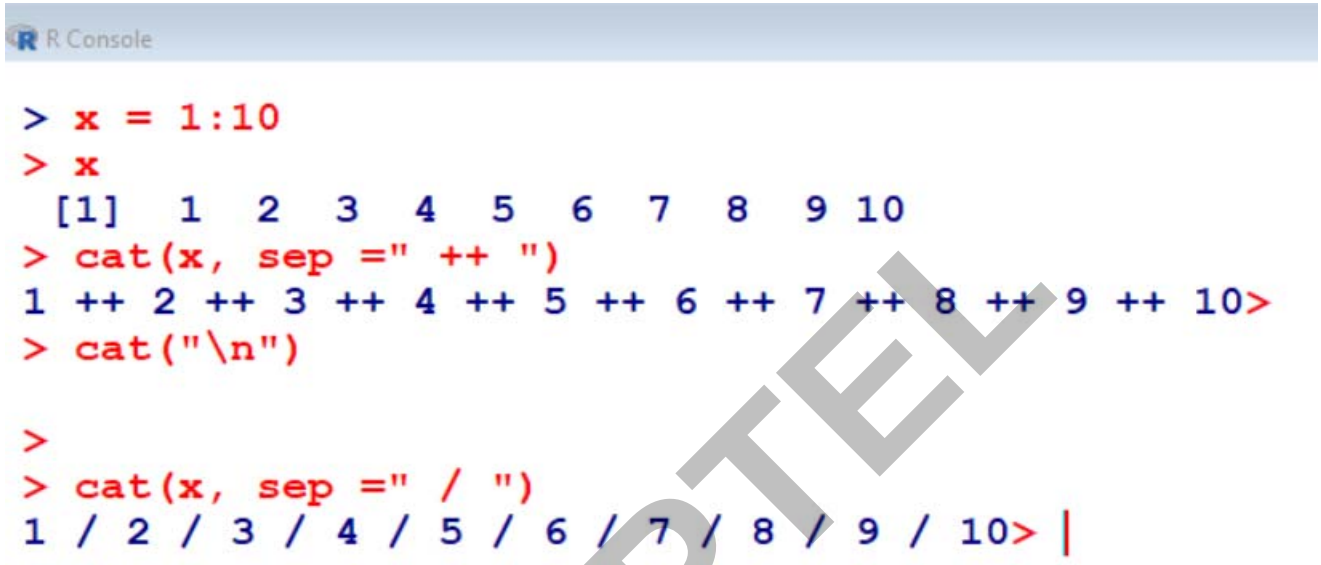
> cat(x, sep =" ++ ")
1 ++ 2 ++ 3 ++ 4 ++ 5 ++ 6 ++ 7 ++ 8 ++ 9 ++ 10

> cat("\n")   # This is used to change the line

> cat(x, sep =" / ")
1 / 2 / 3 / 4 / 5 / 6 / 7 / 8 / 9 / 10
```

NPTEL

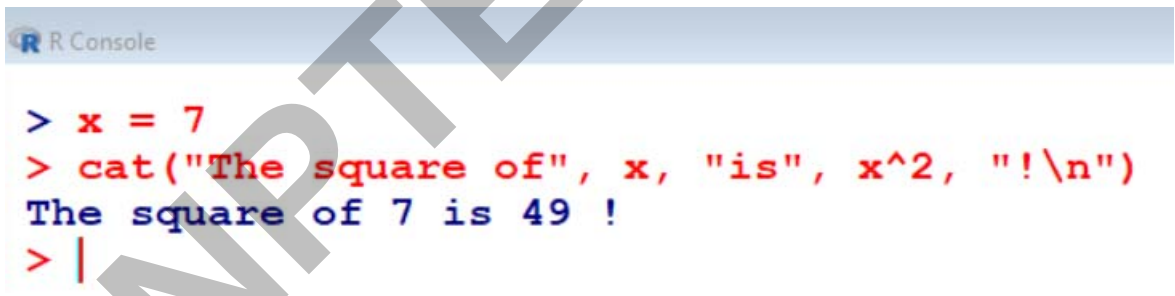
Formatting and Display of Strings



NPTEL

Formatting and Display of Strings

```
> x = 7
> cat("The square of", x, "is", x^2, "!\n")
The square of 7 is 49 !
```

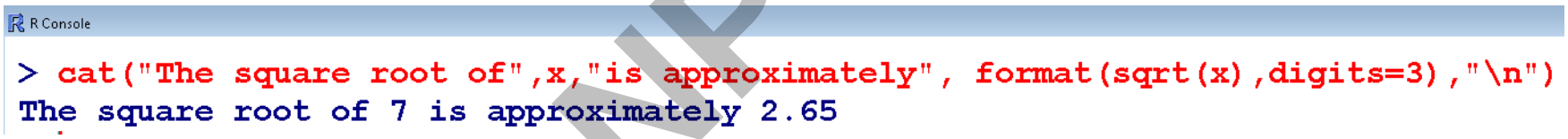


NPTEL

Formatting and Display of Strings

```
> cat("The square root of",x,"is
approximately", format(sqrt(x),digits=3),"\n")

The square root of 7 is approximately 2.65
```



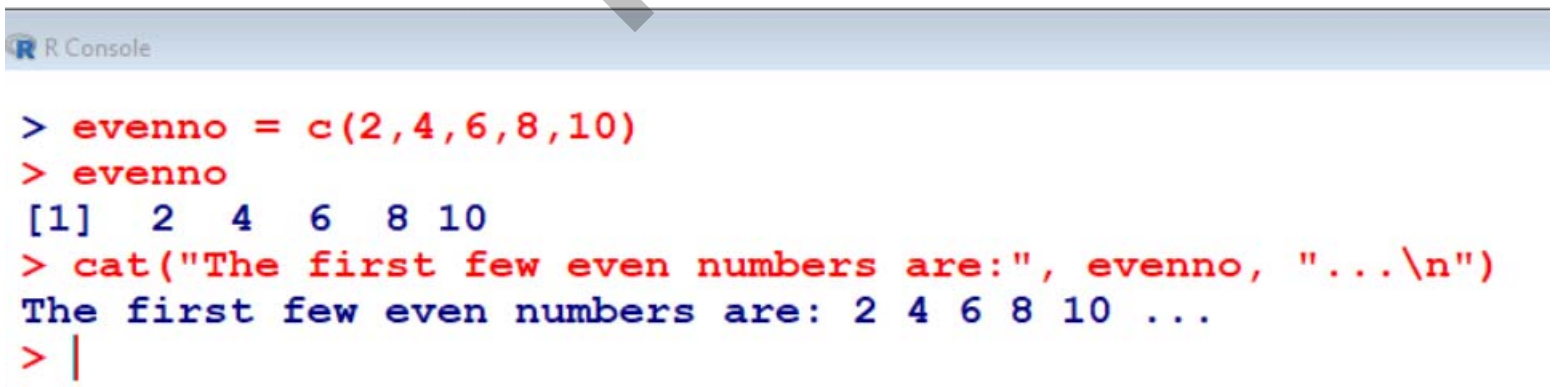
NPTEL

Formatting and Display of Strings

The `cat` function can also print simple vectors

```
> evenno = c(2,4,6,8,10)
> evenno
[1]  2  4  6  8 10

> cat("The first few even numbers are:",
evenno, "...\n")
The first few even numbers are: 2 4 6 8 10 ...
```



NPTEL

Formatting and Display of Strings

```
> x = 1:10
> x
 [1]  1  2  3  4  5  6  7  8  9 10

> cat(x, fill = 2, labels = paste("(", letters[1:10], "):"))

( a ): 1
( b ): 2
( c ): 3
( d ): 4
( e ): 5
( f ): 6
( g ): 7
( h ): 8
( i ): 9
( j ): 10
```

NPTEL

Foundations of R Software

Lecture 37

Strings – Display and Formatting

:::

Paste Function

Shalabh

Department of Mathematics and Statistics

Indian Institute of Technology Kanpur

NPTEL

Formatting and Display of Strings: `paste`

- The `paste()` function concatenates several strings together.

- It creates a new string by joining the given strings end to end.

- The result of `paste()` can be assigned to a variable

  (in contrast to the function `cat()`).

NPTEL

Formatting and Display of Strings

Usage

```
paste (..., sep = " ", collapse = NULL).
```

`collapse` is an optional character string to separate the results.

- The parameter `sep` is a string that serves as a separation between the strings that are given as input.
- `paste` inserts a single space between pairs of strings.
- A desired line break can be achieved with `"\n"` (newline).

NPTEL

Formatting and Display of Strings: `paste`

- `paste` converts its arguments to character strings (via `as.character`) , and concatenates them (separating them by the string given by `sep`).

- If the arguments are vectors, they are concatenated term-by-term to give a character vector result.

- If a value is specified for `collapse`, the values in the result are then concatenated into a single string, with the elements being separated by the value of `collapse`.

NPTEL

Formatting and Display of Strings: `paste`

- The `paste()` function concatenates several strings together.

- It creates a new string by joining the given strings end to end.

- The result of `paste()` can be assigned to a variable (in contrast to the function `cat()`).

NPTEL

`paste0(..., collapse)`

is equivalent to `paste(..., sep = "", collapse)`,

slightly more efficiently.

Formatting and Display of Strings

Example: `paste` converts its arguments to character strings (via `as.character`), and concatenates them

```
> paste(1:12)
 [1] "1"  "2"  "3"  "4"  "5"  "6"  "7"  "8"
"9"  "10" "11" "12"

> as.character(1:12) #Alternative to paste
 [1] "1"  "2"  "3"  "4"  "5"  "6"  "7"  "8"
"9"  "10" "11" "12"
```

NPTEL



Formatting and Display of Strings

```
> paste("Everybody", "loves", "R Programming.")
[1] "Everybody loves R Programming."

> paste("Everybody", "loves", "R Programming.",
sep="*")
[1] "Everybody*loves*R Programming."

> paste("Everybody", "loves", "R Programming.",
sep="===")
[1] "Everybody===loves===R Programming."
```

NPTEL

Formatting and Display of Strings

R Console

```
> paste("Everybody", "loves", "R Programming.")
[1] "Everybody loves R Programming."
>
> paste("Everybody", "loves", "R Programming.", sep="*")
[1] "Everybody*loves*R Programming."
>
> paste("Everybody", "loves", "R Programming.", sep="===")
[1] "Everybody===loves===R Programming."
```

NPTEL

Formatting and Display of Strings

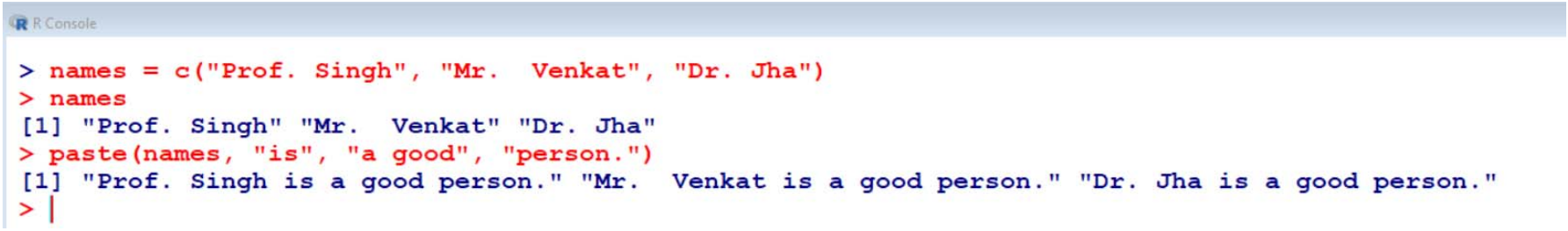
If one or more arguments are vectors of strings, `paste` will generate all combinations of the arguments:

```
> names = c("Prof. Singh", "Mr.  Venkat", "Dr. Jha")

> names
[1] "Prof. Singh" "Mr.  Venkat" "Dr. Jha"

> paste(names, "is", "a good", "person.")
[1] "Prof. Singh is a good person." "Mr.  Venkat is a good person." "Dr. Jha is a good person."
```

NPTEL



Formatting and Display of Strings

When we want to join even those combinations into one, big string.

The `collapse` parameter defines a top-level separator and instructs `paste` to concatenate the generated strings using that separator:

```
> names = c("Prof. Singh", "Mr.  Venkat", "Dr. Jha")

> paste(names, "is", "a good", "person.", collapse=", and ")
[1] "Prof. Singh is a good person., and Mr. Venkat is a good person., and Dr. Jha is a good person."
```

NPTEL

Formatting and Display of Strings

R Console

```
> names = c("Prof. Singh", "Mr.  Venkat", "Dr. Jha")
>
> paste(names, "is", "a good", "person.", collapse=", and ")
[1] "Prof. Singh is a good person., and Mr.  Venkat is a good person., and Dr. Jha is a good person."
>
```

NPTEL

Operations with Strings

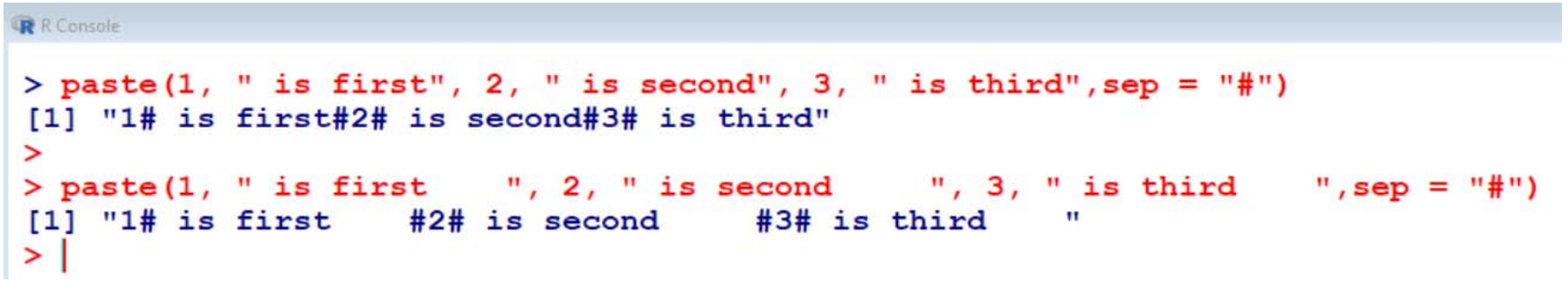
Example:

```
> paste(1, " is first", 2, " is second", 3, " is third",
sep = "#")
[1] "1# is first#2# is second#3# is third"
```

Observe the role of blank space inside double quotes

```
> paste(1, " is first    ", 2, " is second     ", 3, " is
third    ", sep = "#")
[1] "1# is first    #2# is second     #3# is third    "
```

NPTEL



Operations with Strings

Example:

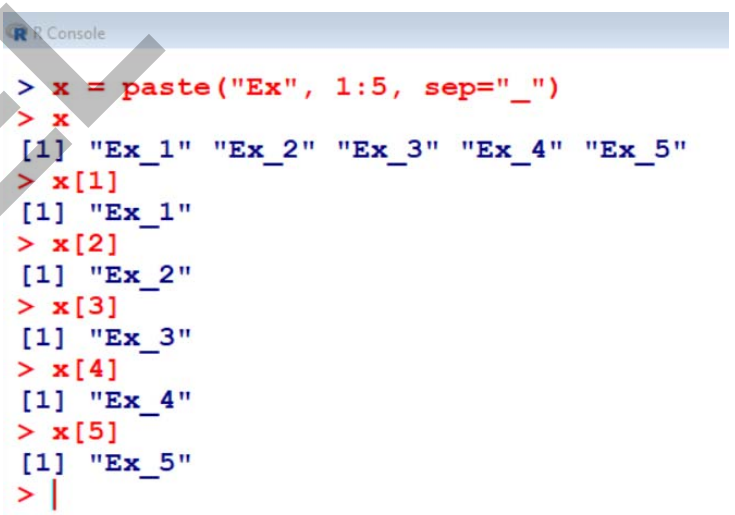
```
> x = paste("Ex", 1:5, sep="_")
>x
[1] "Ex_1" "Ex_2" "Ex_3" "Ex_4" "Ex_5"

> x[1]
[1] "Ex_1"

> x[2]
[1] "Ex_2"

> x[3]
[1] "Ex_3"

> x[5]
[1] "Ex_5"
```



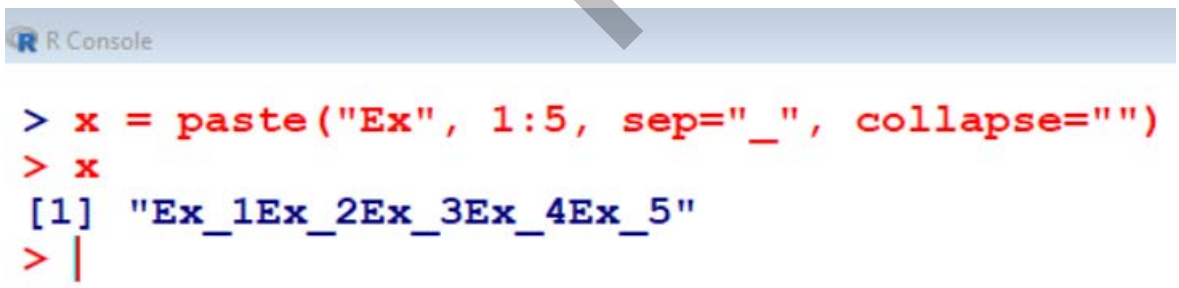
NPTEL

Operations with Strings

x is a vector of strings.

If we use the parameter `collapse`, a single string, instead of a vector of strings, is created:

```
> x = paste("Ex", 1:5, sep="_", collapse="")
> x[1]
[1] "Ex_1Ex_2Ex_3Ex_4Ex_5"
```



NPTEL

Operations with Strings

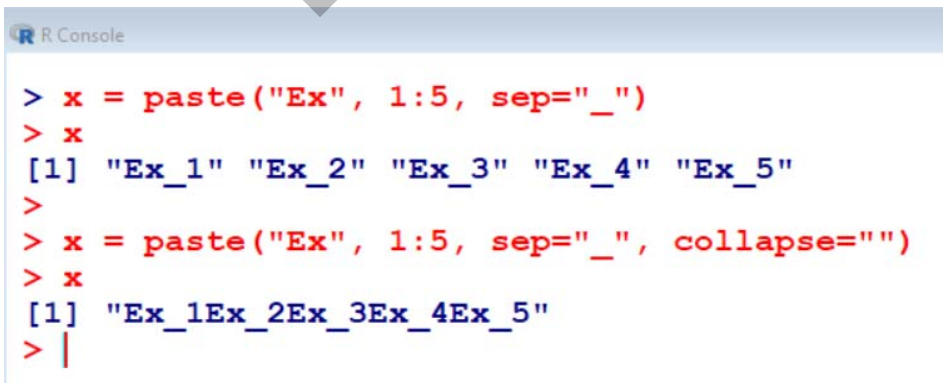
Note the difference between

```
x = paste("Ex", 1:5, sep="_")
> x
[1] "Ex_1" "Ex_2" "Ex_3" "Ex_4" "Ex_5"
```

and

```
x = paste("Ex", 1:5, sep="_", collapse="")
> x
[1] "Ex_1Ex_2Ex_3Ex_4Ex_5"
```

NPTEL



Operations with Strings

When using a single vector, `paste0` and `paste` have the same outcome and they work the same `as.character`.

```
> paste0(1:10)
 [1] "1"  "2"  "3"  "4"  "5"  "6"  "7"  "8"
"9"  "10"

> paste(1:10)
 [1] "1"  "2"  "3"  "4"  "5"  "6"  "7"  "8"
"9"  "10"
```

NPTEL



Operations with Strings

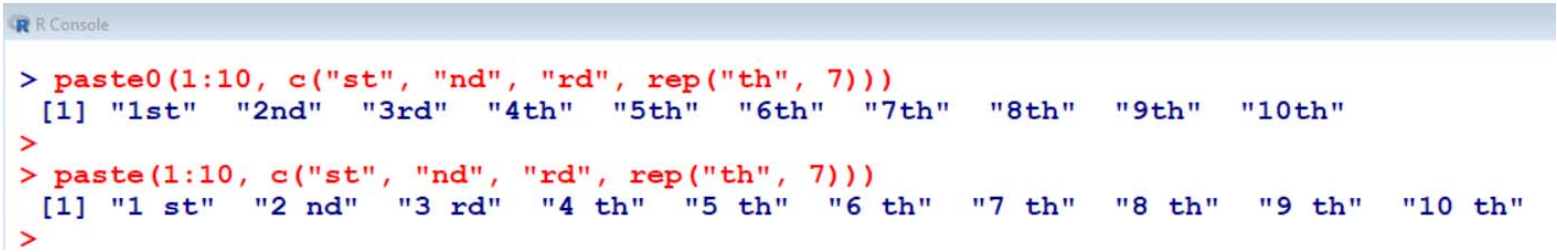
When we use more than one vectors to `paste0` then they concatenate in a vectorized way.

```
> paste0(1:10, c("st", "nd", "rd", rep("th", 7)))
 [1] "1st"  "2nd"  "3rd"  "4th"  "5th"  "6th"  "7th"
"8th"  "9th"  "10th"
```

Observe the role of blank space

```
> paste(1:10, c("st", "nd", "rd", rep("th", 7)))
 [1] "1 st"  "2 nd"  "3 rd"  "4 th"  "5 th"  "6 th"
"7 th"  "8 th"  "9 th"  "10 th"
```

NPTEL



Foundations of R Software

Lecture 38

Strings – Display and Formatting

:::

String Splitting

Shalabh

Department of Mathematics and Statistics

Indian Institute of Technology Kanpur

NPTEL

Operations with Strings

Command `strsplit`, split the elements of a character vector.

"Split" can be a single character, or a character string:

Usage

```
strsplit(x, split, fixed = FALSE, …)
```

Arguments

`x`	character vector, each element of which is to be split.
`split`	character vector containing regular expression(s) (unless fixed = TRUE) to use for splitting.
`fixed`	logical. If `TRUE` match split exactly, otherwise use regular expressions.

NPTEL

Operations with Strings

- With a command `strsplit`, we can split a string in pieces.

```
> x = "The&!syntax&!of&!paste&!is!&available!
&inthe online-help"

> x
[1] "The&!syntax&!of&!paste&!is!&available!
&inthe online-help"

> strsplit(x, split="!")
[[1]]
[1] "The&"              "syntax&"    "of&"
[4] "paste&"            "is"         "&available"
[7] " &inthe online-help"
```

NPTEL

Operations with Strings

R Console

```
> x = "The&!syntax&!of&!paste&!is!&available! &inthe online-help"
> x
[1] "The&!syntax&!of&!paste&!is!&available! &inthe online-help"
> strsplit(x, split="!")
[[1]]
[1] "The&"                  "syntax&"              "of&"
[4] "paste&"                "is"                   "&available"
[7] " &inthe online-help"
```

NPTEL

Operations with Strings

```
> x = "The&!syntax&!of&!paste&!is!&available!
&inthe online-help"
> x
[1] "The&!syntax&!of&!paste&!is!&available!
&inthe online-help"

> strsplit(x, split = "&!")
[[1]]
[1] "The"
[2] "syntax"
[3] "of"
[4] "paste"
[5] "is!&available! &inthe online-help"
```

NPTEL

Operations with Strings

R Console

```
> x = "The&!syntax&!of&!paste&!is!&available! &inthe online-help"
> x
[1] "The&!syntax&!of&!paste&!is!&available! &inthe online-help"
> strsplit(x, split = "&!")
[[1]]
[1] "The"
[2] "syntax"
[3] "of"
[4] "paste"
[5] "is!&available! &inthe online-help"
```

NPTEL

Operations with Strings

```
> x = "The&!syntax&!of&!paste&!is!&available!
&inthe online-help"


> x
[1] "The&!syntax&!of&!paste&!is!&available!
&inthe online-help"

> y = strsplit(x, split = "!&")

> y
[[1]]
[1] "The&!syntax&!of&!paste&!is"    "available!
&inthe online-help"
```

NPTEL

Operations with Strings

Notice the access to single components:

```
> y[[1]][1]
[1] "The&!syntax&!of&!paste&!is"

> y[[1]][2]
[1] "available! &inthe online-help"

> y[[1]][3]
[1] NA
```

NPTEL

Operations with Strings

R Console

```
> x = "The&!syntax&!of&!paste&!is!&available! &inthe online-help"
> x
[1] "The&!syntax&!of&!paste&!is!&available! &inthe online-help"
> y  =  strsplit(x, split = "!&")
> y
[[1]]
[1] "The&!syntax&!of&!paste&!is"    "available! &inthe online-help"

> y[[1]][1]
[1] "The&!syntax&!of&!paste&!is"
> y[[1]][2]
[1] "available! &inthe online-help"
> y[[1]][3]
[1] NA
> |
```

NPTEL

Operations with Strings

Example: Convert to matrix

```
> dates = c("2020-07-24", "2021-08-25", "2022-
09-26", "2023-10-27")
```

Split the dates

```
> datesplt  = strsplit(dates, "-")
> datesplt
[[1]]
[1] "2020" "07"   "24"

[[2]]
[1] "2021" "08"   "25"

[[3]]
[1] "2022" "09"   "26"

[[4]]
[1] "2023" "10"   "27"
```

NPTEL

Operations with Strings

Example: Convert to matrix

Create matrix of dates and outcome is that the elements are character

```
> datemat = matrix(unlist(datesplt), nrow = 4,
ncol=3, byrow=TRUE)

> datemat
     [,1]   [,2] [,3]
[1,] "2020" "07" "24"
[2,] "2021" "08" "25"
[3,] "2022" "09" "26"
[4,] "2023" "10" "27"
```

NPTEL

Operations with Strings

Example: Convert to matrix

Create matrix of dates and outcome is that the elements are numbers

```
> datematrix =
matrix(as.numeric(unlist(datesplt)), nrow = 4,
ncol=3, byrow=TRUE)

> datematrix
     [,1] [,2] [,3]
[1,] 2020    7   24
[2,] 2021    8   25
[3,] 2022    9   26
[4,] 2023   10   27
```

NPTEL

Operations with Strings

Example: Convert to matrix

R Console

```
> dates = c("2020-07-24", "2021-08-25", "2022-09-26", "2023-10-27")
> dates
[1] "2020-07-24" "2021-08-25" "2022-09-26" "2023-10-27"
> datesplt  = strsplit(dates, "-")
> datesplt
[[1]]
[1] "2020" "07"   "24"

[[2]]
[1] "2021" "08"   "25"

[[3]]
[1] "2022" "09"   "26"

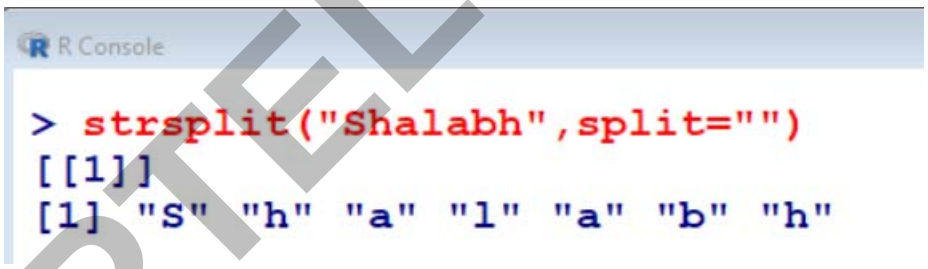
[[4]]
[1] "2023" "10"   "27"

> datemat = matrix(unlist(datesplt), nrow = 4, ncol=3, byrow=TRUE)
> datemat
     [,1]   [,2] [,3]
[1,] "2020" "07" "24"
[2,] "2021" "08" "25"
[3,] "2022" "09" "26"
[4,] "2023" "10" "27"
> datematrix = matrix(as.numeric(unlist(datesplt)), nrow = 4, ncol=3, byrow=TRUE)
> datematrix
     [,1] [,2] [,3]
[1,] 2020    7   24
[2,] 2021    8   25
[3,] 2022    9   26
[4,] 2023   10   27
>
```

NPTEL

Operations with Strings

```
> strsplit("Shalabh",split="")
[[1]]
[1] "S" "h" "a" "l" "a" "b" "h"
```



NPTEL

Foundations of R Software

Lecture 39

Strings – Display and Formatting

:::

Manipulations with Strings and Alphabets

NPTEL

Shalabh

Department of Mathematics and Statistics

Indian Institute of Technology Kanpur

Operations with Strings: Counting characters

nchar takes a character vector as an argument and returns a vector whose elements contain the sizes of the corresponding elements of x.

nzchar is a fast way to find out if elements of a character vector are non-empty strings.

NPTEL

Operations with Strings: Counting characters

Usage

```
nchar(x, type = "chars", allowNA = FALSE,
keepNA = NA)

nzchar(x, keepNA = FALSE)
```

Arguments

`x` character vector, or a vector to be coerced to a character vector.

`type` character string: partial matching to one of `c("bytes", "chars", "width")`.

`keepNA` logical: should `NA` be returned when `x` is `NA`?

NPTEL

Operations with Strings: Counting characters

There are a variety of commands that can be used for strings.

Example:

Count of number of characters:

```
> x = "R course 24.07.2022"

> y = "Number of participants: 25"


> nchar(x) #Count the Number of Characters in x
[1] 19

> nchar(y) #Count the Number of Characters in y
[1] 26
```

NPTEL

Operations with Strings: Counting characters

Example:

The `nzchar(x)` function takes the character vector `x` as a parameter. Its output is either `TRUE` or `FALSE`.

```
> x = "R course 24.07.2022"

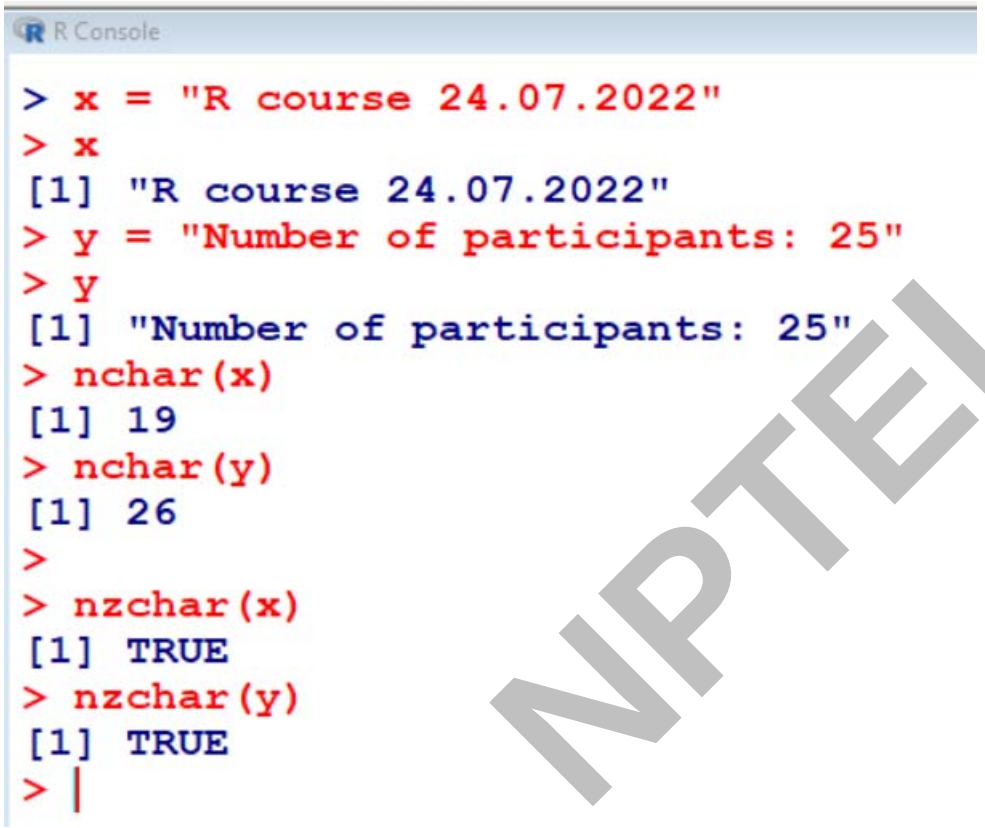
> y = "Number of participants: 25"


> nzchar(x)
[1] TRUE

> nzchar(y)
[1] TRUE
```

NPTEL

Operations with Strings: Counting characters



NPTEL

Operations with Strings: Counting characters

Example: `nzchar(x)`

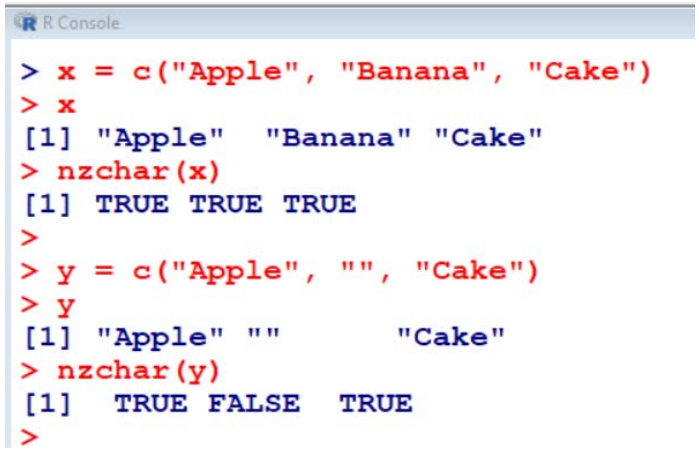
```
> x = c("Apple", "Banana", "Cake")
> x
[1] "Apple"  "Banana" "Cake"

> nzchar(x)
[1] TRUE TRUE TRUE

> y = c("Apple", "", "Cake")
> y
[1] "Apple" ""      "Cake"

> nzchar(y)
[1]  TRUE FALSE  TRUE
```

NPTEL



Operations with Strings: Counting characters

Example:

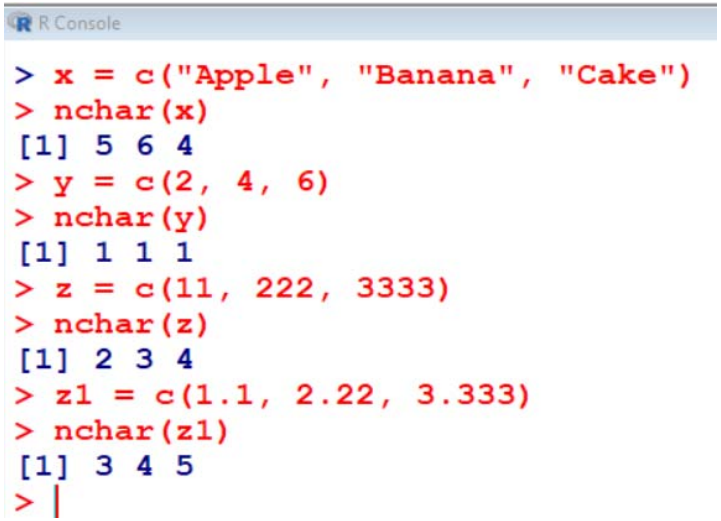
```
> x = c("Apple", "Banana", "Cake")
> nchar(x)
[1] 5 6 4

> y = c(2, 4, 6)
> nchar(y)
[1] 1 1 1

> z = c(11, 222, 3333)
> nchar(z)
[1] 2 3 4

> z1 = c(1.1, 2.22, 3.333)
> nchar(z1)
[1] 3 4 5
```

NPTEL



Operations with Strings: Lower and upper cases

`tolower(x)` and `toupper(x)` Functions:

`tolower(x)` and `toupper(x)` convert upper-case characters in a character vector to lower-case, or vice versa.

Non-alphabetic characters are left unchanged.

NPTEL

Operations with Strings: Lower and upper cases

`tolower(x)` and `toupper(x)` Functions:

Examples:

```
> x = "R course will start from 24.07.2022"

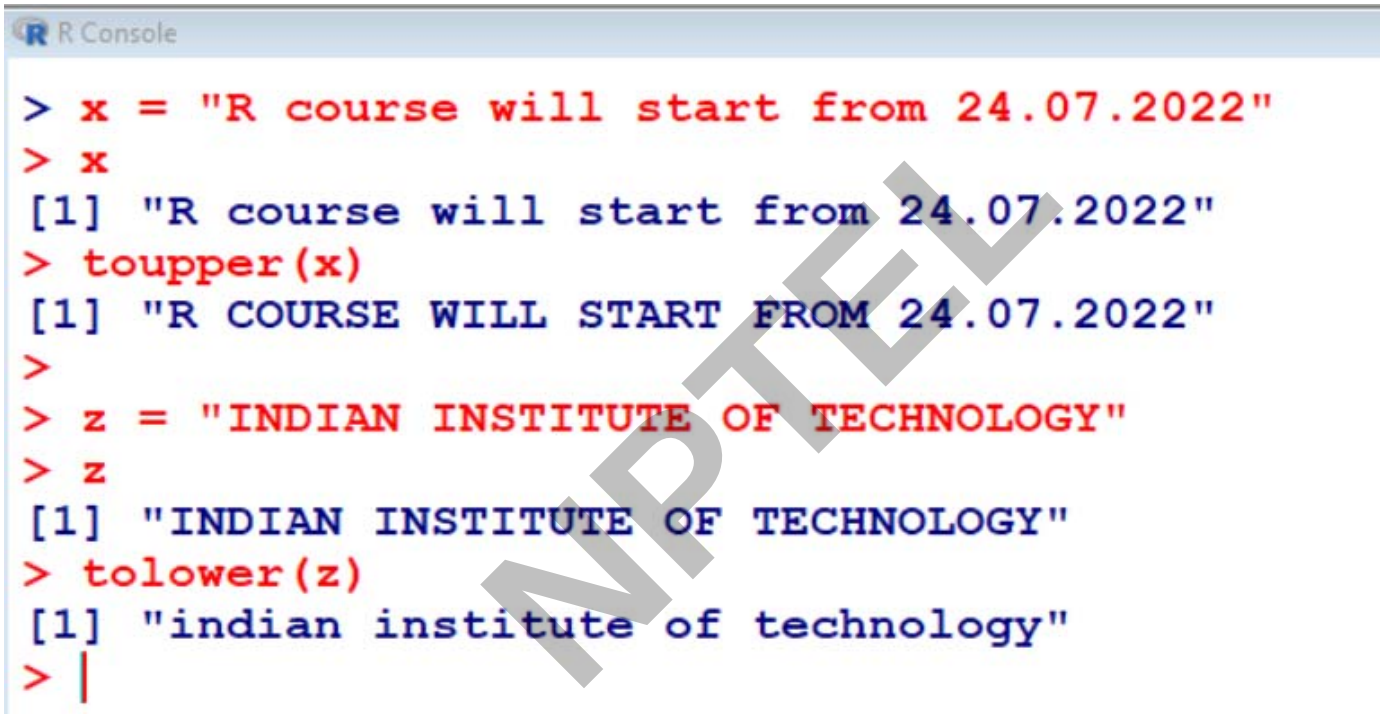
> toupper(x)
[1] "R COURSE WILL START FROM 24.07.2022"

> z = "INDIAN INSTITUTE OF TECHNOLOGY"

> tolower(z)
[1] "indian institute of technology"
```

NPTEL

Operations with Strings: Lower and upper cases



NPTEL